# CS141 Coursework Two - Report

Archisha Biswas - 5689456

This document is a short report detailing the features of, and the design choices made in, my submission for the second coursework for CS141 Functional Programming.

## Overall Structure

### Combining `State` and `IO`

The `main` function runs the program by calling the `runREPL` function in `TUI.hs`. This function is of the type `StateT (Map String String) IO ()`, rather than just `IO ()`, to combine the `State` and `IO` monads by building a monad transformer stack. This is used to thread memory throughout the program, as required by Part Three of the specification. The entire main REPL is therefore one large stateful computation, executed using runStateT. To facilitate this, the `respondTo` function returns a value of the type `StateT (Map String String) IO ()` - this function makes use of stateful values and sometimes updates these, depending on the request the user has made to the chatbot. Using this monad transformer stack also required me to change line 113 of `Spec.hs` to utilise the `evalStateT` function to execute the REPL and retrieve the output to the user, discarding the final state - doing so allows for the tests written for the coursework to still function given the monad transformer stack.
The current value of the state is however still passed around the program from function to function, as the `parseArithmetic` function, defined within the `parseRequest` function, requires access to the contents of the state, but is not part of the monad transformer stack. For more detail, see Part Three.

### The `Request` Data Type

The REPL defined in `TUI.hs` is carried out in two broad sections - parsing the user's input, as done with the `readRequest` and `parseRequest` functions, then responding to this input appropriately using the `respondTo` function. These functions all make use of the `Request` data type. This type has many different constructors, which are used to create different `Request` values depending on the user input that has just been parsed - the different constructors can store different types of data that have just been parsed from the user, which can then be used when responding to the user. These constructors then give the `respondTo` function an easy way to respond to different inputs with appropriate outputs - it pattern-matches on `Request` values by constructor.

## Part One

To complete this section of the coursework, I used the `time` library, both when finding the current day of the week, and when finding the difference between this and a user-inputted date. `Data.Day.Time` includes the newtype `Day`, a value of which I create to store user-inputted dates, as well as the `diffDays` function to find the difference between days. It also includes the `dayOfWeek` function which converts a `Day` to a `DayOfWeek` - this type has an `Enum` instance, allowing for use of the `succ` function to go from one day of the week to the next.

## Part Two

For this section of the coursework, I used the `makeExprParser` function from `Control.Monad.Combinators.Expr`. This function builds an arithmetic parser and evaluator from a parser used to evaluate individual terms (in this case longhand integers - I used the `parseLonghand` function provided), and an operator table, where the fixity of each operator is defined. This is what led me to deciding to use this function, as trying to write a left-recursive parser (required to parse operations left-to-right) by hand was proving difficult. As the program is only dealing with integers, I decided to also add `mod` and `div` to the operator table, meaning that queries such as "What is fifty mod three?" result in the correct answer. The table also distinguishes between infix and prefix operations, so I decided to add "negative" as an operator to indicate negative numbers - as long as the overall answer is positive, requests such as "What is three times negative eight plus seventy?" give the correct output.

## Part Three

As described above, I added memory to the chatbot by combining `State` and `IO` using a monad transformer stack. The state holds a `Map` of values, with the name of items being the keys and their respective things to remember the values - using a `Map` allows for quick look-ups when checking if the program currently remembers information about a given item. However, the state contents are still passed around the program separately. This is because `makeExprParser` both parses arithmetic expressions, and evaluates them. This means that when the user enters the word "that" to refer to a previously calculated value, the parser parsing the word must return this previously calculated value to be used in the evaluation - this requires access to the current contents of the state, which `parseRequest` would not have without passing this separately. This is unideal, but is forced by the evaluation of arithmetic expressions also happening within the parser itself, rather than in the `respondTo` function.
I also added the ability for the chatbot to forget things it currently knows - this is done by replacing the state with an empty `Map` if everything should be forgotten, or by using `put` to alter the state and delete the key in the `Map` if a specific value should be forgotten. This ability can be tested by entering the request "Forget everything.", or "Forget about [name]".

## Beyond the Specification

I decided to add the ability for the chatbot to respond to requests of the form "When is the next tram from [station] to [terminus]?", i.e. giving the number of minutes until the next tram heading towards a specified terminal stop arrives at a specified station. I did this using the Transport for Greater Manchester Metrolink API, which provides the live data shown on each Passenger Information Display (PID) across the network. I decided to make GET requests using the `Wreq` library, as this provides functions to easily make requests with headers (which I needed to pass my authenticator key), as well as the ability to make GET requests without concretely defining the shape of the data to be received, which I was finding difficult with the format of the API data. I used a variety of functions from across `Data.Aeson`, `Data.Char`, `Data.Foldable` and `Data.Text` to convert JSON values to usable Haskell types, as well as `Network.Wreq` to make the initial request.
Here are some examples of inputs that should result in successful responses most of the time (bar delays and construction works) - "When is the next tram from Old Trafford towards Altrincham?"; "When is the next tram from Wharfside towards Crumpsall?"; "When is the next tram from Baguley towards Manchester Airport?". For other potential inputs, please consult the Tram Map, linked below.

## Resources

Edwards, Stephen. 2020. "Haskell: Monads. A 5-Minute Introduction." *YouTube*. YouTube. https://www.youtube.com/watch?v=_Gk_lwhJMzk.

"Monad Transformers — Monday Morning Haskell." 2019. *Monday Morning Haskell*. https://mmhaskell.com/monads/transformers.

"Send a GET Request with Headers in Haskell." 2016. *Stack Overflow*. https://stackoverflow.com/questions/36145056/send-a-get-request-with-headers-in-haskell.
"Simple StateT Use - HaskellWiki." 2007. *HaskellWiki*. https://wiki.haskell.org/Simple_StateT_use.
"Tram Map | Bee Network | Powered by TfGM." 2026. https://tfgm.com/ways-to-travel/tram/network-map.
"WREQ/WWW/Tutorial.md at Master · Haskell/WREQ." 2018. *GitHub*. https://github.com/haskell/wreq/blob/master/www/tutorial.md.
Yao, William. 2019. "A Cheatsheet to the Time Library." https://williamyaoh.com/posts/2019-09-16-time-cheatsheet.html.